Theme Report: Observe

1. Theme

Observe in embedded systems is about the method of taking in information from the outside world so that the system can gather useful data to make decisions with. Observation is really important because it defines how the system interprets the environment, and only based on that can it make any type of decision. Therefore, capturing precise and realistic data is a core step for an embedded system.

Observation involves bridging hardware and software using tools like Analog-to-Digital converters (as most sensors give analog data), sensors, transducers (to convert different types of energy into electrical energy), filters (to ensure data fits a valid range), and other acquisition modules. An example would include converting sound waves passed through to a microphone by converting them to a voltage using transducer, filtering out extreme low and high frequencies (like with capacitors), applying a constant DC offset, and scaling the voltage before passing it to an ADC. The combination of these modules lets the embedded systems convert inputs like sudden changes in current/voltage into useful information that a micro-controller can act upon.

Since systems require observed data, its the observation phase that directly affects behaviour. Sampling rate, signal stability, noise, and hardware configuration impact how well the system perceives its environment. Poor observation leads to incorrect system interpretation, even if later processing stages function correctly.

2. Background

To expand upon the earlier paragraph, the latter phases, reason and act, are very much dependent on the observation phase not just because without data there is nothing to analyze and act upon, but messy data can lead to unpredictable behaviour. Consider a modern car with advanced sensors. Suppose the forward camera becomes partially blocked by dirt, rain, or glare from sunlight. Due to this, lanes appear blurry, proximity to nearby cars could be completely off, and pedestrians might not be seen. This can lead to sudden braking because a shadow is interpreted as an obstacle and drifting in nearby lanes. This is not due to the rest of the system not performing correctly, but rather the poor data quality captured by the camera is misinforming the sensors that control speed and keep the car driving straight.

Embedded systems perform observation using peripherals like GPIO ports and Analog-to-Digital converters. Using these, they can detect both absolute voltage values (high vs low level) and transitions (rising vs falling). The more precise observation is, the more likely system behavior matches desired behavior as its using high-quality data.

Embedded systems design usually begins by putting together hardware interfaces that can accurately receive signal transmissions. This can involve hardware components like pull-up resistors to avoid a floating voltage (by providing a stable path to 3v3), mathematical considerations like following Nyquist theorem during sampling, and technical considerations like noting resolution limits when using

ADCs for Analog-to-Digital signal conversion. Setting clear constraints can help improve perception and ultimately minimize error between expected and actual results.

The earlier labs (1-3) focused on observation while increasing in complexity. The first lab focused on only detecting button presses and accounting for de-bouncing while using GPIO ports. This was the most basic form of observation with polling: wait for button press with short delay, then light up LED. The next lab built on detecting button press but with state machines, so the output (LED lights up) was not constant but rather dependent on state. Finally, the last lab involved detecting analog waveforms and converting them into digital data using an ADC. Together, these activities demonstrate how a real-world system can capture a spectrum of data, from simple button pushes to analog (and possibly ‘messy’) signals more characteristic of true sensor data.

3. Theme Exemplars

Exemplar 1: Digital Signal Detection Using GPIO (Lab 1)

Method

The system was configured to observe a push-button input using a GPIO pin. Prior to using the pull-up, the LED would sometimes flicker or not blink when expected to. Upon adding a pull-up resistor, I noticed the input remained at a defined logic level when the button was not pressed. When pressed, the input transitioned to logic low, which the program detected and used to control an LED output. GPIO registers were configured to set direction, enable digital function, and read input state.

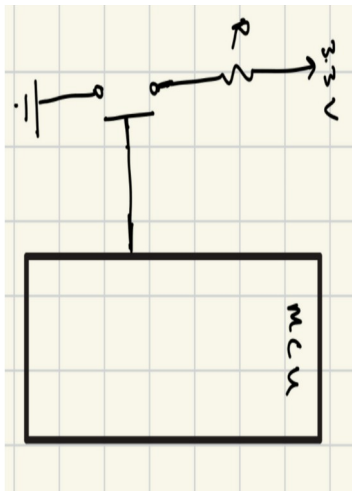
Validation Results

I validated the behavior by pressing and releasing the button 20 times while logging LED response.

- Button pressed ==> input detected ==> LED ON
- Button released ==> input idle ==> LED OFF

Correct behaviour confirmed that the micro-controller could reliably observe discrete digital events. The use of a pull-up resistor prevented floating inputs, ensuring stable signal detection. The photos below show the diagram of pull-up resistor and the excel table.

Figure 1: Pull-up resistor schematic



Case	Pull-Up Re	Button Pre	Pin Reads	LED State
1	Yes	No	HIGH (1)	ON
2	Yes	Yes	LOW (0)	OFF
3	Yes	No	HIGH (1)	ON
4	Yes	Yes	LOW (0)	OFF
5	Yes	No	HIGH (Stat	ON
6	No	No	Floating	Random/Unstable
7	No	Yes	LOW (0)	OFF
8	No	No	Floating	Random/Unstable
9	No	Yes	LOW (0)	OFF
10	No	No	Floating	Random/Unstable

Figure 2: Trials with and without pull-up resistor

Exemplar 2: Analog Signal Acquisition Using ADC (Lab 3)

Method

An analog voltage from the AD3 was applied to a pin on the Cortex-M. The ADC was configured to sample the signal and convert it into digital values stored in memory. My team was assigned the 80 Hz signal frequency, so I first tried 160 Hz sampling rate since Nyquist theorem says >= 2*input_frequency. However, I noticed that the exact shape wasn't captured for the square waveform. The square wave form looked more triangular than square. Then I increased frequency to 10 times, so 800 Hz and got a very clean graph. Both graphs (at 160 and 800 Hz) are pasted below.

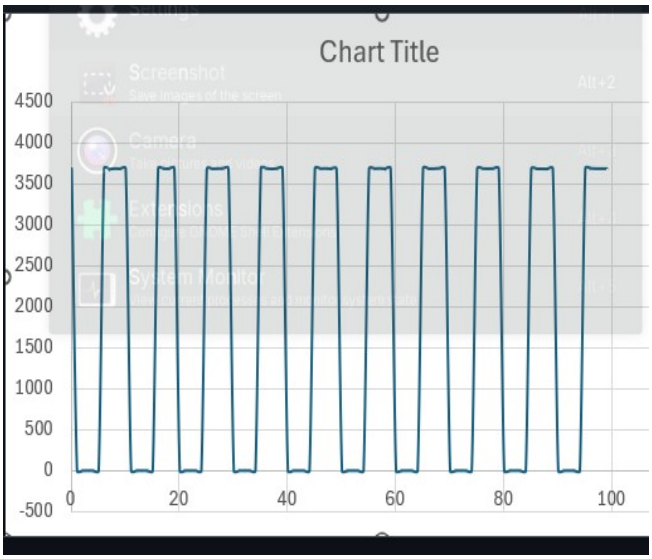


Figure 3: Sampling square wave at 800 Hz

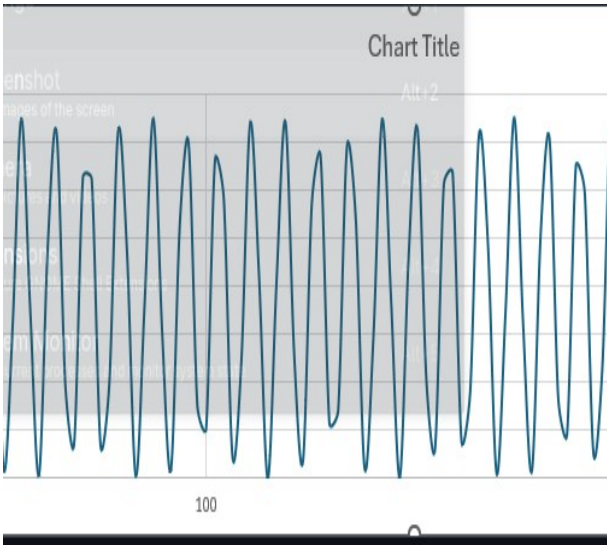


Figure 4: Sampling square wave at 160 Hz

Validation Results

Validation involved comparing measured digital values and plotted waveforms to expected signal characteristics:

- DC inputs produced stable digital readings proportional to voltage.
- Sampling at twice the frequency was not enough to preserve shape
- Sampling at ten times the frequency led to accurate re-creation of the waveform
- Square wave sampling showed discrete transitions consistent with input switching.

These results confirmed correct analog observation and digital representation of time-varying signals.

4. Debugging Exemplar

During the Observe theme labs, I used different debugging approaches depending on the lab and the scenario. Some milestones were more software-oriented so debugging involved more register-inspection, while other milestones involved using the AD3 to measure voltages and verify they were in the correct range.

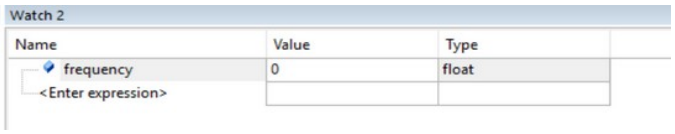
Assembly-Level Debugging Using Register Inspection

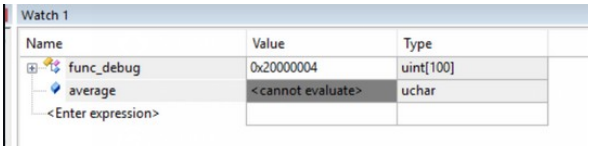
In the digital signal and finite state machine labs, I used Thumb assembly and Keil. Because there was a lot of register manipulation, I found setting breakpoints and stepping through loops, conditionals, etc was really useful during execution to explain the behavior.

I used breakpoints and inspected register values to verify:

- GPIO initialization values written to the correct hardware registers
- bit manipulation and masking operations
- conditional branching behaviour

I found that by seeing how register values changed at each breakpoint, I could tell whether instructions produced the expected hardware state. For example, during lab 3 I noticed the “average” variable wasn’t updating, this was the average of the n samples in the func_debug array. I wasn’t sure if this was due to hardware (wrong connection/signal) or software so I used debugger to check for software. I expanded the func_debug variable as shown in the photo and saw that all the values were printed. This told me that the hardware was collecting inputs, but for some reason, “average” was showing as “cannot update”. Therefore, the issue had to be software. I checked my code and noticed that I had defined a second variable “mean” and was accidentally using that. That is why “average” showed up as “cannot update”. The debugger helped me recognize where the issue really was.





Functional Debugging During ADC Signal Acquisition

For analog signal acquisition and the frequency counter bonus task, debugging required observing real-time sampled data without interrupting execution. Pausing the processor would interfere with timing-dependent measurements, so functional debugging techniques were used.

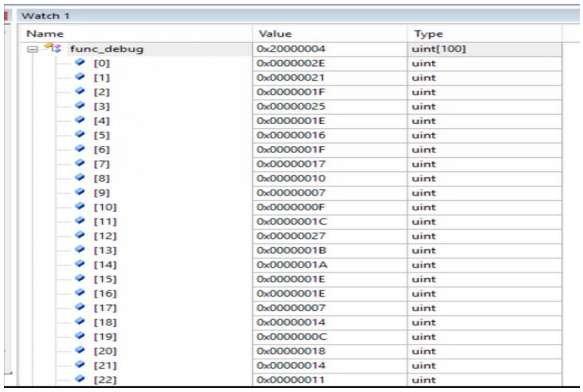
A global memory array stored ADC samples collected at a fixed sampling rate controlled by the SysTick timer. These stored values were inspected using the debugger or exported for analysis. Runtime validation included:

- tracking minimum and maximum ADC values to determine signal range
- computing the DC offset from observed extrema

- detecting midpoint crossings to determine waveform period
- verifying sampling timing from system clock and delay configuration

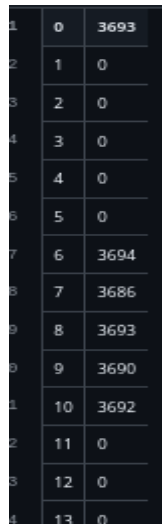
LED toggling was also used as a simple real-time indicator of program execution without halting the system.

This approach confirmed that the ADC sampling process, timing configuration, and frequency calculations were operating correctly.

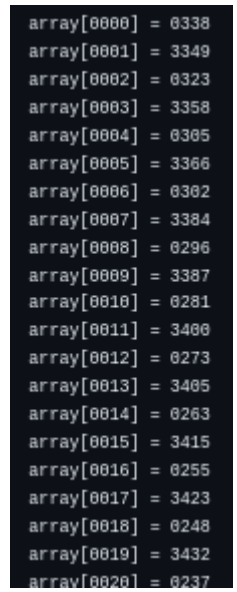


Watch 1

Name	Value	Type
func_debug	0x20000004	uint[100]
[0]	0x0000002E	uint
[1]	0x00000021	uint
[2]	0x0000001F	uint
[3]	0x00000025	uint
[4]	0x0000001E	uint
[5]	0x00000016	uint
[6]	0x0000001F	uint
[7]	0x00000017	uint
[8]	0x00000010	uint
[9]	0x00000007	uint
[10]	0x0000000F	uint
[11]	0x0000001C	uint
[12]	0x00000027	uint
[13]	0x0000001B	uint
[14]	0x0000001A	uint
[15]	0x0000001E	uint
[16]	0x0000001E	uint
[17]	0x00000007	uint
[18]	0x00000014	uint
[19]	0x0000000C	uint
[20]	0x00000018	uint
[21]	0x00000014	uint
[22]	0x00000011	uint



1	0	3693
2	1	0
3	2	0
4	3	0
5	4	0
6	5	0
7	6	3694
8	7	3686
9	8	3693
0	9	3690
1	10	3692
2	11	0
3	12	0
4	13	0



```
array[0000] = 0338
array[0001] = 3349
array[0002] = 0323
array[0003] = 3358
array[0004] = 0305
array[0005] = 3366
array[0006] = 0302
array[0007] = 3384
array[0008] = 0296
array[0009] = 3387
array[0010] = 0281
array[0011] = 3400
array[0012] = 0273
array[0013] = 3405
array[0014] = 0263
array[0015] = 3415
array[0016] = 0255
array[0017] = 3423
array[0018] = 0248
array[0019] = 3432
array[0020] = 0237
```

Hardware-Level Debugging Using the Analog Discovery

In addition to software-based debugging, physical signal verification was performed using the Analog Discovery measurement tools. This allowed direct observation of voltage levels in the circuit to confirm correct electrical behaviour. For example, during lab 3, using the AD3 I was able to ensure there was indeed a voltage drop generated due to the input frequency.

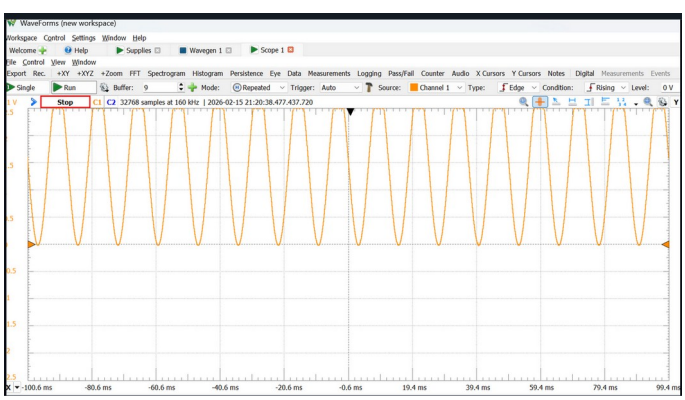


Figure 6: Measuring voltage using AD3

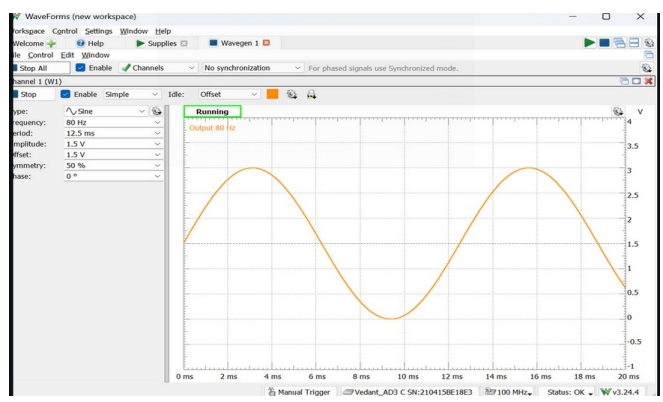


Figure 5: Generating voltage using AD3

5. Synthesis

The examples I have given above illustrate the observe theme by showing how embedded systems get input information from the environment via a variety of ways like button presses and analog signals. These can be verified using a mix of hardware and software systems.

Digital input detection using GPIO illustrates observation of discrete events. The system determines whether a defined electrical state exists and responds accordingly. Reliable observation depends primarily on signal stability, which is ensured through hardware design choices such as pull-up resistors and debouncing techniques.

Analog signal acquisition using the ADC illustrates observation of continuous physical phenomena. Here, the system must not only detect a signal but also sample and represent it accurately over time. This introduces additional design constraints, including sampling frequency, resolution, and signal conditioning. The ADC exemplar shows that even when a signal is detected, insufficient sampling quality can distort how the system represents that signal.

Overall, they show that observation is not just sensing the environment but also setting up the hardware correctly, using software debugging software, interpreting data appropriately, and paying attention to timing and frequency design. Embedded systems use this first phase, the observe phase, to provide the remaining phases with accurate data that represents the useful information in the environment. Using peripherals, this allows for the transformation of physical events into useful digital input that may be further processed or reasoned upon. Poorly collected data during observe phase would force the next phases to use inaccurate data, with possibly severe consequences in system behaviour.

From this, we can see the key role of observation in embedded systems. Observation defines the how well “good” information is present for the future processing. This allows for system behaviour to better match real conditions. When it is not done well, the rest of the system is acting upon erroneous data, even if those later phases are individually engineered very well.

6. Reflection

a) Understanding of the Observe Theme

The exemplars mentioned demonstrate how observation in embedded systems is more complex than just taking data input, and involves signal filtering, hardware configurations, and careful debugging. Observation might sound like the most passive stage, but in reality it involves active measurement design. I learned that measurement needs to be validated carefully, as its not automatically correct.

b) Understanding of Intelligent Systems and Microprocessors

The labs demonstrated how microprocessors interact with their environment using peripherals like GPIO and ADCs. These labs taught me how real-world signals have to be processed in order to be turned into useful information for the digital ports on the microcontroller. Debugging and measurement tools showed me that system perception depends on both electrical behaviour and software configuration. I also learned how timing, resolution, and hardware configuration influence system perception.

7. References

[1] J. W. Valvano, *Embedded Systems: Introduction to ARM Cortex-M Microcontrollers*, 5th ed. CreateSpace Independent Publishing Platform, 2019.

[2] MSP432E401Y Microcontroller Datasheet and Technical Reference Manual.

[3] Laboratory Manual — Digital Signals, Finite State Machines, and Analog Signal Acquisition.